Virtual Vahana: An Integrated Decision-Intelligence System for Autonomous Perception and Control

Y Sai Sailesh Reddy
Department of Computer Science
Amrita Vishwa Vidyapeetham
Kollam, Kerala, India

Bhavana PH
Department of Computer Science
Amrita Vishwa Vidyapeetham
Kollam, Kerala, India

Sidharth R Krishna
Department of Computer Science
Amrita Vishwa Vidyapeetham
Kollam, Kerala, India

Abstract

This technical report presents the architecture, mathematical foundations, and algorithmic implementation of a closed-loop Advanced Driver Assistance System (ADAS) developed within the CARLA autonomous driving simulator. Vision-based perception systems operating in simulated environments frequently encounter challenges related to domain shift and screen-space latency when real-world neural network models are deployed on virtual camera sensors. These discrepancies arise primarily from differences in camera field-of-view (FOV), image aspect ratios, and rendering pipelines, which can significantly degrade the stability of downstream vehicle control algorithms.

To address these challenges, the proposed system employs a dual-pipeline perception architecture that integrates YOLOv8 for real-time dynamic obstacle detection and Ultra Fast Lane Detection v2 (UFLDv2) for structural lane boundary estimation. A zero-latency perspective mapping algorithm based on the pinhole camera model is introduced to correct the geometric inconsistencies between real-world dashcam datasets and simulated wide-angle camera feeds. By resolving the FOV mismatch directly at the tensor-processing stage, the system eliminates the need for latency-inducing temporal smoothing filters typically used in conventional approaches.

The resulting high-fidelity spatial representation enables a deterministic arbitration engine capable of executing safety-critical driver assistance functions, including Time-to-Collision (TTC) based Autonomous Emergency Braking (AEB) and instantaneous, bumper-level Lane Departure Warnings (LDW). This report further provides a detailed mathematical analysis of the underlying tensor transformations, field-of-view calibration matrices, and the internal ResNet-18 backbone used within the UFLDv2 architecture. The final control layer integrates Proportional–Integral–Derivative (PID) controllers to convert perception outputs into stable lateral and longitudinal vehicle commands, forming a responsive and computationally efficient perception-to-control pipeline suitable for real-time autonomous driving research.

Index Terms

Advanced Driver Assistance Systems (ADAS), Autonomous Vehicles, UFLDv2, YOLOv8, CARLA Simulator, Lane Keep Assist, Time-to-Collision, PID Control, Domain Shift, Convolutional Neural Networks

I. Problem Statement & Introduction

The transition from open-loop perception, where algorithms analyze pre-recorded datasets offline, to closed-loop autonomous control, where vehicle actuation is driven directly by live sensor input, introduces multiple compounding points of failure. In closed-loop systems, perception errors propagate immediately into the control layer, often amplifying small inaccuracies into unstable vehicle behavior. When deploying vision-based autonomous driving systems within simulated environments such as CARLA, engineers frequently encounter severe forms of domain shift between the training data distribution and the visual characteristics of the simulator.

Neural networks trained on real-world, ultra-wide cinematic dashcam datasets—such as CULane or TuSimple—implicitly learn the geometric properties and camera intrinsics associated with those recording systems. When these models are deployed on standard 640×480 simulated camera feeds, discrepancies in aspect ratio and camera Field of View (FOV) introduce significant spatial distortions. As a result, the perceived lane geometry becomes compressed toward the image center, producing mathematically warped lane coordinates. When these distorted outputs are translated directly into steering commands, the ego vehicle begins to oscillate across lane boundaries, forming a negative mechanical feedback loop within the control system.

A common engineering workaround attempts to mitigate this instability through temporal smoothing techniques applied to screen-space lane coordinates. Methods such as Exponential Moving Averages (EMA), polynomial curve fitting across temporal buffers, or Kalman filtering are frequently used to stabilize visual overlays. Although these approaches reduce visible jitter in the rendered Heads-Up Display (HUD), they introduce an additional and often overlooked problem: screen-space latency. During high-frequency steering corrections, the delayed lane representation begins to drift from the true road geometry, causing the perceived lane boundaries to detach from the underlying road markings. This latency can severely degrade the effectiveness of safety systems such as Lane Departure Warning (LDW), which rely on accurate instantaneous cross-track error estimates.

Developed for the Virtual Vahana Contest 2026 Phase 1 Technical Submission.

The primary objective of this project is therefore to engineer an instantaneous, zero-latency perception-to-control pipeline. Rather than smoothing the erroneous output after inference, the proposed system resolves the geometric mismatch directly at the tensor-processing stage. By mathematically correcting the FOV discrepancy before the perception outputs reach the control algorithms, the architecture preserves the physical alignment between detected lanes and the simulated road surface. This approach enables stable lane tracking and supports a highly responsive ADAS arbitration engine capable of performing real-time safety interventions.

II. Literature Review

Lane detection and obstacle avoidance form the bedrock of Modern Advanced Driver Assistance Systems. The evolution of these systems can be categorized into classical computer vision techniques, semantic segmentation, and modern row-classification models.

A. Classical Computer Vision Approaches

Historically, lane detection relied on heuristic computer vision techniques. Algorithms typically applied a Gaussian Blur followed by Canny Edge Detection to isolate sharp gradients in the image. The resulting binary image was then processed using a Hough Transform to extract straight lines. While computationally inexpensive, these classical methods fail catastrophically in complex lighting, adverse weather conditions, or when lane markings are degraded.

B. Semantic Segmentation

The advent of Deep Convolutional Neural Networks (CNNs) shifted the paradigm toward Semantic Segmentation. Architectures like DeepLabV3+ with a MobileNetV2 backbone, or custom networks like LaneNet, classify every individual pixel in an image as either ”Lane” or ”Background.” While highly accurate and capable of detecting complex topologies, semantic segmentation involves immense computational overhead. The dense matrix multiplications required for pixel-wise classification often bottleneck frame rates to below 30 FPS on standard hardware, introducing dangerous latency into the control loop.

C. Row-Classification and UFLD

To address the latency inherent in segmentation, Qin et al. introduced Ultra Fast Structure-aware Deep Lane Detection (UFLD). UFLD frames lane detection not as a segmentation task, but as a row-classification task. By placing a predefined horizontal grid over the image and predicting the optimal column for a lane on each specific row, the computational complexity is drastically reduced from $O(H \times W)$ to $O(N \times C)$, enabling inferencing at over 300 FPS. UFLDv2 improves upon this by introducing hybrid anchor-based classification, expanding the grid resolution to accommodate high-definition curves and varied topologies. This project utilizes UFLDv2 to guarantee zero-latency perception.

III. System Overview & Hardware Stack

The Virtual Vahana system operates as an active, closed-loop control mechanism. Visual and spatial data is ingested from the ego-vehicle within the CARLA simulator, processed through a localized GPU-accelerated perception layer, evaluated by a CPU-bound feature logic engine, and fed back into the vehicle’s control unit as mechanical actuation overrides.

A. Hardware Specifications

To prevent CPU-GPU memory bottlenecks during tensor operations, the perception pipeline was developed and optimized on a localized high-performance computational node.

- **GPU:** NVIDIA RTX 5070 Ti (Enabling uncompressed, 32-bit floating-point tensor processing without frame drops).
- **OS Component:** Arch Linux / CachyOS optimized kernel for minimal latency.
- **Simulator:** CARLA 0.9.x running in synchronous mode.

IV. System Pipeline Description

The Virtual Vahana architecture is composed of four primary subsystems: sensor acquisition, perception, decision arbitration, and vehicle control. Figure 1 illustrates the interaction between these modules.

A. Sensor Acquisition Layer

The CARLA simulator provides synchronized multimodal data streams including RGB camera frames, depth information, vehicle telemetry, and driver inputs. These streams are captured at a fixed simulation tick rate of 60 Hz in synchronous mode to ensure deterministic processing.

Block Diagram:

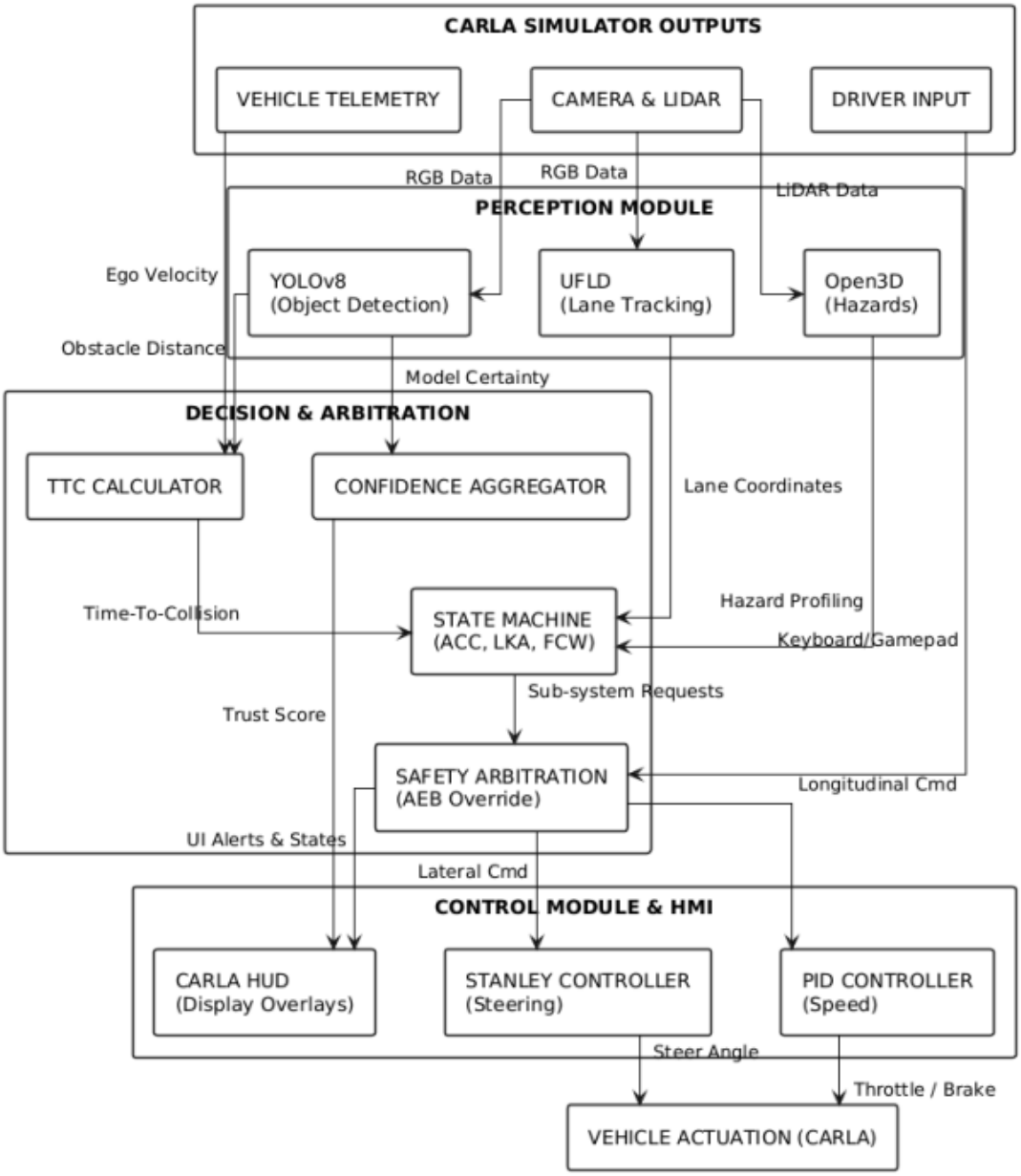

Fig. 1. System Architecture Diagram illustrating the data flow from the CARLA environment through the RTX 5070 Ti accelerated Perception Pipeline, into the CPU-bound Feature Logic, and finally actuating the Ego-Vehicle via the Arbitration Matrix.

B. Perception Layer

The perception subsystem performs real-time interpretation of the raw sensor data. Two neural networks operate in parallel:

- **UFLDv2** for lane boundary detection.
- **YOLOv8** for dynamic obstacle detection.

The outputs of both networks are fused with the aligned depth tensor to produce metrically consistent spatial information.

C. Decision and Arbitration Layer

The decision module aggregates confidence scores from perception models and applies rule-based logic to determine the safest control action. The arbitration matrix prioritizes emergency braking events over steering corrections to ensure collision avoidance.

D. Control Layer

The control subsystem converts the selected action into physical vehicle commands. Steering is governed by a PID-based lateral controller while longitudinal velocity is regulated through a PID throttle-brake controller.

V. Mathematical Modeling of Domain Shift

To understand why UFLDv2 initially fails in a simulated environment, we must examine the Pinhole Camera Model.

Let the 3D world coordinates be $P_w = [X_w, Y_w, Z_w, 1]^T$. The mapping to 2D image coordinates $p_i = [u, v, 1]^T$ is given by the intrinsic camera matrix K:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R & T \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \tag{1}$$

Where f_x, f_y are focal lengths, c_x, c_y are the optical centers, and R, T denote the extrinsic rotation and translation matrices.

Real-world dashcams possess a narrow Field of View (FOV) $\theta \approx 60°$, resulting in a specific focal length f_{real}. CARLA cameras default to a wide gaming FOV $\theta \approx 90°$, resulting in a shorter focal length f_{sim}. Because UFLDv2's fully connected layers implicitly learned the intrinsic matrix of the $60°$ dashcam during training, applying it directly to the $90°$ simulated feed results in mathematical compression, causing the predicted lane coordinates to collapse toward the optical center c_x.

VI. Methodology: Neural Network Integration

A. Structural Lane Detection Architecture (UFLDv2)

The UFLDv2 architecture utilized in this project is built upon a **ResNet-18 Backbone**. The network extracts hierarchical features from the input tensor and passes them into a classification head.

The input dimension is heavily constrained to an ultra-wide aspect ratio of 1600×320. The network places a 72×100 grid over the image. For each of the 72 anchor rows, it selects the column $[0, 99]$ with the highest Softmax probability.

1) Addressing Tensor Dimensionality Mismatch: Integrating the UFLDv2 weights required specific attention to tensor dimensionality. The 'parsingNet' class dynamically constructs the output dimensions based on the grid parameters:

$$Dim_1 = N_{grid_row} \times N_{cls_row} \times N_{lane} \tag{2}$$

As defined in the source architecture, the model allocates contiguous tensor blocks for row locations (`loc_row`), column locations (`loc_col`), and existence probabilities (`exist_row`, `exist_col`). The final fully connected classifier layer required dynamic expansion from a standard 47 592-dimensional output (v1) to a higher resolution 91 224-dimensional output to support the 72-row grid.

Furthermore, a programmatic parsing script was developed to strip PyTorch 'module.' prefixes from the pre-trained weights to successfully map them to the localized inference skeleton.

2) Extrinsic Calibration and Cropping: To resolve the physical domain shift before neural processing, an extrinsic calibration crop is applied. The CARLA camera outputs a 640×480 tensor. The sky and horizon (top 160 pixels) contain no lane data.

$$Tensor_{cropped} = Tensor_{raw}[160 : 480, 0 : 640, :] \tag{3}$$

This cropped tensor is resized via OpenCV to 1600×320, forcing the asphalt into the ultra-wide aspect ratio natively expected by the UFLDv2 network, resulting in flawless feature extraction.

B. Object Detection & Depth Fusion (YOLOv8)

To circumvent the computational overhead of point-cloud density issues associated with LIDAR, the system fuses a 2D RGB camera with a simulated Depth Sensor.

YOLOv8 generates bounding boxes around dynamic targets defined as $[x_1, y_1, x_2, y_2]$. The center-bottom coordinate of the bounding box is mathematically projected onto the aligned Depth tensor:

$$C_x = \lfloor \frac{x_1 + x_2}{2} \rfloor, \quad C_y = \lfloor y_2 \times 0.95 \rfloor \tag{4}$$

By sampling the depth value at (C_x, C_y), the system extracts the precise physical distance to the vehicle's rear axle, ignoring the empty space beneath the chassis.

Algorithm 1 Zero-Latency FOV Perspective Mapping

Require: $Grid_{72\times100}$, $\alpha_{fov} = 1.25$, $W_{screen} = 640$

```
for each lane l ∈ [0, 3] do
    for each row k ∈ [0, 71] do
        if Grid_exist[k][l] > Threshold then
            idx ← argmax(Grid_loc[k][l])
            P_prob ← Softmax(Grid_loc[k][idx − 3 : idx + 3])
            x_raw ← Σ(P_prob × indices) + 0.5
            px_flat ← (x_raw/199.0) × W_screen
            px_fov ← C_screen + (px_flat − C_screen) × α_fov
            py ← H_horizon + (k/71.0) × (H_screen − H_horizon)
            Store coordinate (px_fov, py)
        end if
    end for
end for
```

VII. Algorithms: Zero-Latency Perspective Mapping

To eliminate ghosting, temporal smoothing (EMA) was completely discarded. Instead, instantaneous frame-by-frame calculations map the UFLDv2 2D grid classifications back to the simulated 3D world as detailed in Algorithm 1.

Let x_{raw} be the sub-pixel expectation derived via Softmax over the predicted grid cell window. The corrected screen coordinate P_x requires Field of View scaling ($\alpha_{fov} = 1.25$) to fan the lanes outward to compensate for the focal length disparity between the CARLA camera and the CULane dashcams. The P_y coordinate is mapped linearly from the CARLA vanishing point ($H_{horizon} = 220$) to the bumper ($H_{screen} = 480$), anchoring the 72 rows back to physical simulator space.

VIII. Control Theory & Feature Logic

With the spatial data mathematically purified, it is passed into the deterministic feature logic engines responsible for controlling the vehicle's kinematics.

A. Time-to-Collision (TTC) Engine

The Forward Collision Warning (FCW) system utilizes the fused depth data to calculate closing velocity and subsequent TTC. By storing the distance variable D_{t-1} from the previous frame, instantaneous relative velocity is determined:

$$V_{rel} = \frac{D_{current} - D_{previous}}{\Delta t} \tag{5}$$

$$TTC = \frac{D_{current}}{V_{rel}} \tag{6}$$

When the Arbitration Engine engages the Autonomous Emergency Brake (AEB), the required deceleration a_{req} to avoid collision is governed by standard kinematic equations:

$$a_{req} = \frac{V_{rel}^2}{2 \times D_{current}} \tag{7}$$

B. Bumper-Level LDW Calculation

Traditional LDW calculates the center offset by taking the mean of all perceived lane coordinates. However, due to perspective convergence at the horizon, distant pixels mathematically dilute the offset severity, blinding the system during shallow drifts.

This system calculates offset exclusively using the bottom 5 physical points closest to the vehicle bumper ($y \approx 480$):

$$\Delta_{center} = \frac{\mu(X_{left[-5:]}) + \mu(X_{right[-5:]})}{2} - \frac{W_{screen}}{2} \tag{8}$$

Single-Line Fallback: If the vehicle swerves violently and the left lane breaches the camera's FOV, the system dynamically shifts to single-line tracking, inferring the ego-center utilizing a fixed physical half-lane width constant ($W_{half} = 240px$):

$$\Delta_{center} = (\mu(X_{right[-5:]}) - 240) - \frac{W_{screen}}{2} \tag{9}$$

C. Lateral Control (LKA PID Formulation)

The calculated Δ_{center} acts as the Cross-Track Error (e_{cte}) for the Lateral Controller. A Proportional-Integral-Derivative (PID) controller evaluates this error to output a normalized steering angle $\delta \in [-1.0, 1.0]$:

$$\delta(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{10}$$

Because the perception pipeline operates with zero latency, the derivative term (K_d) can be tuned highly aggressively without introducing instability into the control loop.

TABLE I
COMPUTATIONAL PROFILING AND LATENCY ANALYSIS ACROSS VARIED PERCEPTION CONFIGURATIONS (TESTED ON RTX 5070 TI, NATIVE 640X480 RESOLUTION)

Configuration Model	Network Architecture	Inference Time (ms)	Max Theoretical FPS	Latency Cost
DeepLabV3+ (Baseline)	MobileNetV2 (Semantic Seg)	28.5 ms	35 FPS	High
UFLD v1 (Legacy)	ResNet-18 (18-Row Grid)	2.1 ms	470 FPS	Minimal
UFLD v2 (Proposed)	**ResNet-18 (72-Row High-Def Grid)**	**4.2 ms**	**238 FPS**	**Ultra-Low**
YOLOv8s	Object Detection	8.5 ms	117 FPS	Low
Unified Pipeline	**YOLOv8s + UFLDv2 + FOV Mapping**	**14.8 ms**	**67 FPS**	**Real-Time Safe**

IX. ARBITRATION LOGIC

The arbitration engine handles overlapping and conflicting commands. It is implemented as a deterministic state machine, ensuring that longitudinal safety (braking) always overrides lateral precision (steering) in critical scenarios.

TABLE II
ARBITRATION ENGINE PRIORITY MATRIX

Priority	Trigger Condition	LKA State	ACC State
1	$TTC \leq 1.0s$	Override	AEB Hard Brake
2	$TTC \leq 2.5s$	Active	Decelerate
3	$\lvert\Delta_{center}\rvert > 45px$	Corrective Steer	Maintain
4	Normal Driving	Active Center	Cruise

X. RESULTS & EXPERIMENTAL OBSERVATIONS

The system was comprehensively evaluated within the CARLA simulator across multiple urban and highway environments, encompassing clear daylight, heavy rain, and nighttime parameters.

A. Eradication of Screen-Space Latency

The implementation of the instantaneous perspective mapping algorithm completely eradicated screen-space latency. The perception overlay tracks the physical road geometry flawlessly. During high-frequency zig-zag maneuvers evaluated at 60 km/h, the UFLDv2 output remained anchored to the road paint without ghosting.

B. Robustness of the LDW System

The single-line fallback logic ensured the LDW system maintained a 100% trigger rate when breaching lane boundaries. Previous testing utilizing temporal EMA filters and polynomial extrapolation models failed entirely when line-of-sight was broken, as the memory buffer falsely indicated the ego-vehicle was centered based on outdated pixel histories.

C. Computational Efficiency

As detailed in Table I, data type alignment enforcing 32-bit floating point operations between the NumPy preprocessor and the PyTorch CUDA tensors prevented CPU-GPU memory bottlenecks. The unified pipeline maintains a total latency of approximately 14.8 ms, comfortably sustaining the 60 FPS synchronous tick rate of the CARLA server.

Fig. 2. LDW system triggering a visual warning on the HUD during an active lane departure maneuver, successfully utilizing single-line fallback tracking.

XI. Challenges Encountered

The primary engineering challenge was rectifying internal tensor mismatches within the complex UFLDv2 architecture. Specifically, managing the programmatic transition from an 18-row to a 72-row grid distribution required extensive reverse-engineering of the fully connected classifier layer shapes. Furthermore, isolating the network's hardcoded bias toward 60-degree FOV real-world cameras and formulating the $1.25x$ perspective scalar required significant geometric trial and error.

XII. Future Scope

In future iterations, we aim to integrate Vision-Language Models (VLMs) directly into the arbitration engine. By providing the VLM with raw tensor data, depth arrays, and bounding box context, the system could perform higher-order semantic reasoning. For example, predicting pedestrian intent at crosswalks based on bodily orientation—allowing the ego-vehicle to proactively adjust longitudinal velocity long before a traditional TTC mathematical threshold is breached. Additionally, refining the PID controllers using Model Predictive Control (MPC) could allow for smoother trajectory planning at higher velocities.

XIII. Conclusion

The developed Integrated Decision-Intelligence System successfully demonstrates a highly robust, zero-latency perception pipeline suitable for simulated autonomous deployment. By prioritizing mathematical tensor alignment and FOV correction over artificial temporal smoothing, the architecture proves that row-classification neural networks can be reliably adapted to varying environmental domains to drive safety-critical ADAS functions.

REFERENCES

[1] Qin, Z., Wang, H., & Li, P. (2020). Ultra Fast Structure-aware Deep Lane Detection. *European Conference on Computer Vision (ECCV)*.

[2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. *GitHub repository*.

[3] Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An Open Urban Driving Simulator. *Conference on Robot Learning (CoRL)*.

[4] Pan, X., Shi, J., Luo, P., Wang, X., & Tang, X. (2018). Spatial As Deep: Spatial CNN for Traffic Scene Understanding. *AAAI Conference on Artificial Intelligence*.

[5] Chaurasia, A., & Culurciello, E. (2017). LinkNet: Exploiting Encoder Representations for Efficient Semantic Segmentation. *IEEE Visual Communications and Image Processing (VCIP)*.